# СЛАУ. Метод Гаусса с выбором главных элементов. Фролов А.А., Курылев Г.А., 2к ИВТ-2

Ссылка на скринкаст: https://disk.yandex.ru/i/4hmLU2MqsTRPJg

## Исходные данные

### Матрицы:

A:

| | | | |
|---|---|---|---|
| 5 | 7 | 6 | 5 |
| 7 | 10 | 8 | 7 |
| 6 | 8 | 10 | 9 |
| 5 | 7 | 9 | 10 |

B:

| |
|---|
| 23 |
| 32 |
| 33 |
| 31 |

### Формулы:

$$\tilde{a}_{ij} = \frac{a_{ij}}{a_{ii}}$$
$$a_{kj} = a_{kj} - \tilde{a}_{ij} * a_{ki}$$

$$i = 1 \div (n-1); k = (i+1) \div n; j = i \div (n+1)$$

$$x_n = \frac{\tilde{a}_{n(n+1)}}{\tilde{a}_{nn}}$$
$$x_i = \tilde{a}_{i(n+1)} - \sum_{j=i+1}^{n} \tilde{a}_{ij} x_j$$
$$i = (n-1) \div 1$$

## Реализация

### Выбор по строкам:

```
def Gauss_row_pivot(A=None, B=None):
```

```python
    if A is None or B is None:
        raise ValueError("A and B must be matrix")

    n = len(A)

    # 1. Формируем расширенную матрицу [A | B]
    for i in range(n):
        A[i].append(B[i])

    # 2. Массив для отслеживания перестановки столбцов (перенумерация переменных)
    col_index = list(range(n))

    # 3. Прямой ход метода Гаусса
    for i in range(n - 1):

        # 3.1 Поиск максимального по модулю элемента в текущей строке
        max_col = i
        for j in range(i + 1, n):
            if abs(A[i][j]) > abs(A[i][max_col]):
                max_col = j

        # 3.2 Перестановка столбцов (если найден элемент больше диагонального)
        if max_col != i:
            for k in range(n):
                A[k][i], A[k][max_col] = A[k][max_col], A[k][i]

            # фиксируем изменение порядка переменных
            col_index[i], col_index[max_col] = col_index[max_col], col_index[i]

        # 3.3 Обнуление элементов под главным элементом
        for k in range(i + 1, n):
            factor = A[k][i] / A[i][i]  # коэффициент исключения
            for j in range(i, n + 1):
                A[k][j] -= factor * A[i][j]

    # 4. Обратный ход (нахождение неизвестных)
    X = [0] * n
    for i in range(n - 1, -1, -1):
        s = A[i][n]
        for j in range(i + 1, n):
            s -= A[i][j] * X[j]
        X[i] = s / A[i][i]

    # 5. Возвращаем переменные в исходном порядке
    result = [0] * n
    for i in range(n):
```

```
        result[col_index[i]] = X[i]

    return result
```

### Выбор по столбцам:

```
def Gauss_column_pivot(A=None, B=None):
    if A is None or B is None:
        raise ValueError("A and B must be matrix")

    n = len(A)

    # 1. Формируем расширенную матрицу
    for i in range(n):
        A[i].append(B[i])

    # 2. Прямой ход
    for i in range(n - 1):

        # 2.1 Поиск максимального элемента в текущем столбце
        max_row = i
        for k in range(i + 1, n):
            if abs(A[k][i]) > abs(A[max_row][i]):
                max_row = k

        # 2.2 Перестановка строк
        if max_row != i:
            A[i], A[max_row] = A[max_row], A[i]

        # 2.3 Обнуление элементов ниже главного
        for k in range(i + 1, n):
            factor = A[k][i] / A[i][i]
            for j in range(i, n + 1):
                A[k][j] -= factor * A[i][j]

    # 3. Обратный ход
    X = [0] * n
    for i in range(n - 1, -1, -1):
        s = A[i][n]
        for j in range(i + 1, n):
            s -= A[i][j] * X[j]
        X[i] = s / A[i][i]

    return X
```

### Выбор по всей матрице:

```
def Gauss_full_pivot(A=None, B=None):
```

```python
    if A is None or B is None:
        raise ValueError("A and B must be matrix")

    n = len(A)

    # 1. Формируем расширенную матрицу
    for i in range(n):
        A[i].append(B[i])

    # 2. Массив для отслеживания перестановки столбцов
    col_index = list(range(n))

    # 3. Прямой ход
    for i in range(n - 1):

        # 3.1 Поиск максимального элемента во всей оставшейся подматрице
        max_row = i
        max_col = i
        max_val = abs(A[i][i])

        for r in range(i, n):
            for c in range(i, n):
                if abs(A[r][c]) > max_val:
                    max_val = abs(A[r][c])
                    max_row = r
                    max_col = c

        # 3.2 Перестановка строк
        if max_row != i:
            A[i], A[max_row] = A[max_row], A[i]

        # 3.3 Перестановка столбцов
        if max_col != i:
            for r in range(n):
                A[r][i], A[r][max_col] = A[r][max_col], A[r][i]

            col_index[i], col_index[max_col] = col_index[max_col], col_index[i]

        # 3.4 Обнуление элементов ниже диагонали
        for k in range(i + 1, n):
            factor = A[k][i] / A[i][i]
            for j in range(i, n + 1):
                A[k][j] -= factor * A[i][j]

    # 4. Обратный ход
    X = [0] * n
    for i in range(n - 1, -1, -1):
        s = A[i][n]
```

```python
        for j in range(i + 1, n):
            s -= A[i][j] * X[j]
        X[i] = s / A[i][i]

    # 5. Возвращаем правильный порядок переменных
    result = [0] * n
    for i in range(n):
        result[col_index[i]] = X[i]

    return result
```

Результат работы:

```
Результат метода Гаусса с выбором главного элемента по строке:
[1.0000000000001554, 0.9999999999999057, 0.9999999999999615, 1.0000000000000229]

Результат метода Гаусса с выбором главного элемента по столбцу:
[0.9999999999999055, 0.9999999999999615, 1.000000000000023, 1.0000000000001554]

Результат метода Гаусса с полным выбором главного элемента:
[0.9999999999999057, 0.9999999999999615, 1.000000000000023, 1.0000000000001554]
```

## Вывод

В ходе работы были реализованы три модификации метода Гаусса: с выбором главного элемента по строкам, по столбцам и по всей матрице.

Все методы приводят систему линейных уравнений к верхнетреугольному виду с последующим выполнением обратного хода для нахождения неизвестных.

Основное различие между методами заключается в способе выбора главного элемента:

- Выбор по строкам требует перестановки столбцов и перенумерации переменных.
- Выбор по столбцам выполняется через перестановку строк и является наиболее удобным и распространённым на практике.
- Полный выбор главного элемента обеспечивает наибольшую численную устойчивость, но требует дополнительных вычислительных затрат.

В результате вычислений для заданной системы все три метода дали одинаковое решение, что подтверждает корректность реализации алгоритмов.